



Sobreviviendo al Caos en Sistemas Distribuidos

- > Tácticas de resiliencia de nivel de producción con Spring Boot 4, Quarkus y Kubernetes.

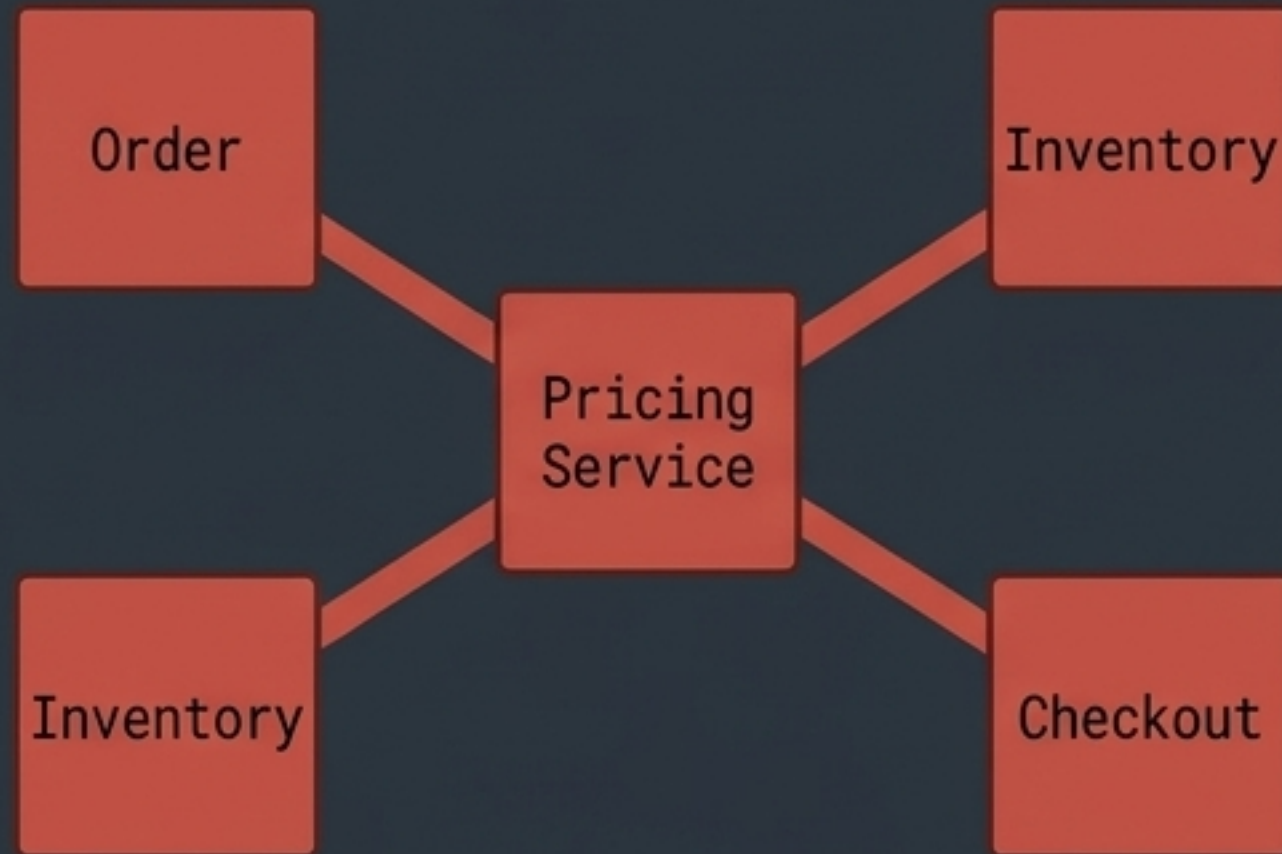
El entorno de red es inherentemente hostil



! Todo lo que puede fallar, fallará. Cada llamada remota es un riesgo. !

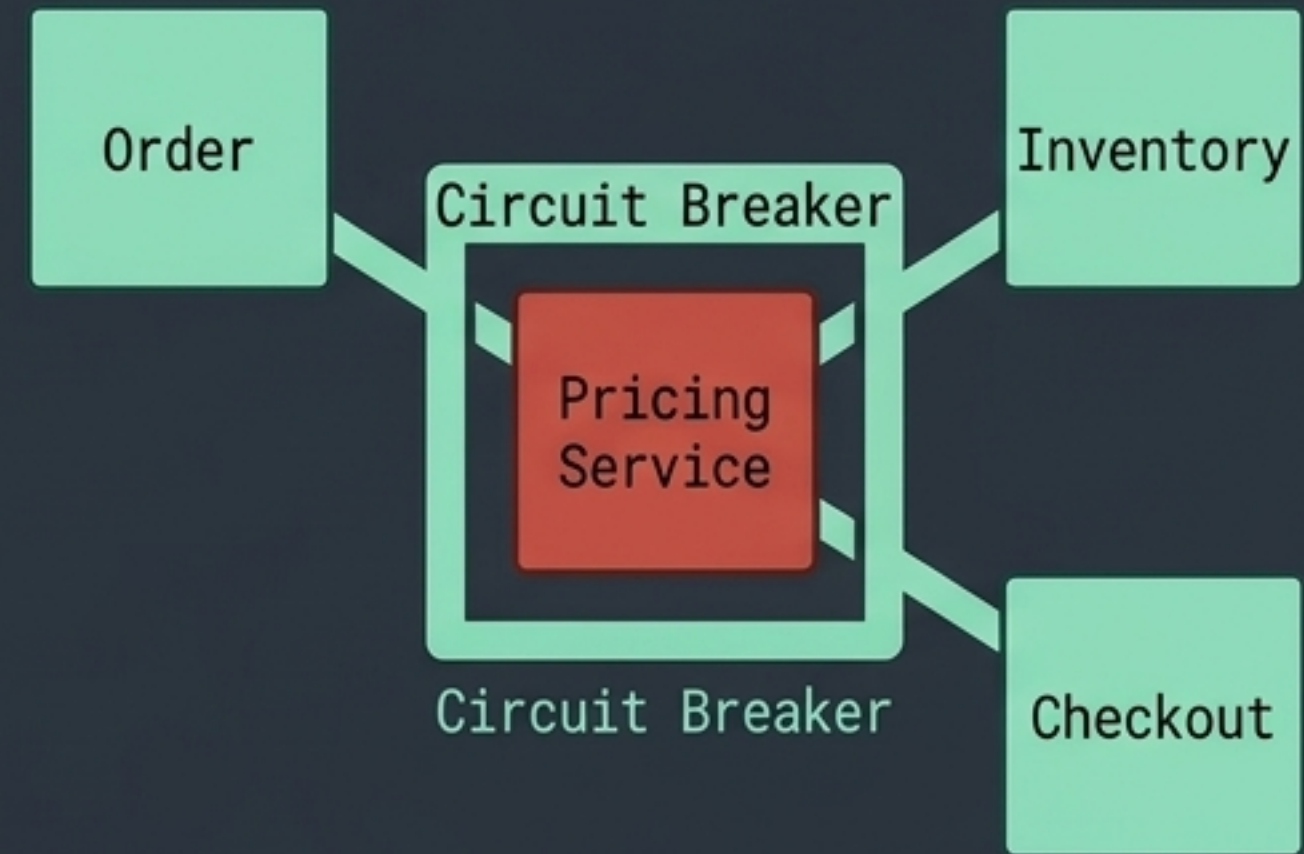
Disponibilidad vs. Resiliencia: Controlando el radio de impacto

Sin Resiliencia



Fallo en cascada. El sistema cae.

Con Resiliencia



Degradación con gracia. El resto opera normal (precio cacheado).

Disponibilidad: % de tiempo que el sistema responde (ej. SLA 99.95%).

Resiliencia: Capacidad de sobrevivir fallos parciales sin caer por completo.

Los 5 escudos fundamentales de la computación distribuida



Timeout

El downstream no responde. (Defiende el latency budget).



Retry

Fallo transitorio (paquete perdido, nodo reiniciando).



Circuit Breaker

El downstream está caído: dejar de gastar recursos.



Bulkhead

Un downstream lento intenta consumir todos nuestros hilos.

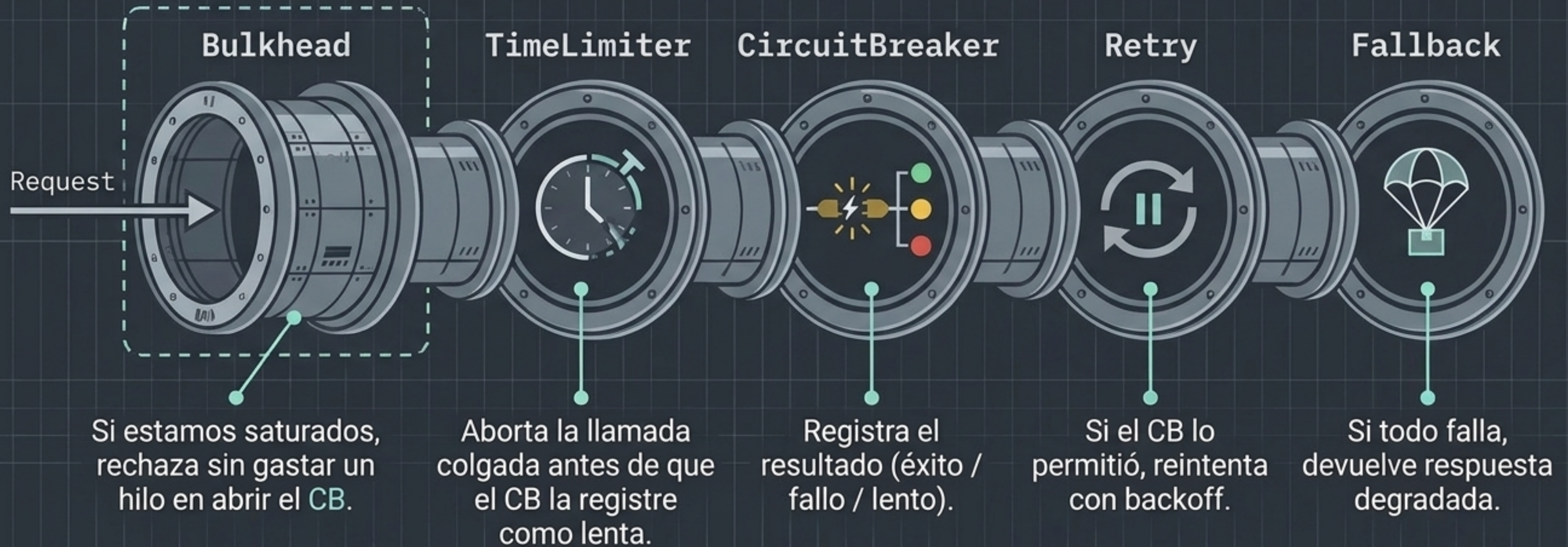


Fallback

Necesitamos una respuesta degradada pero válida.

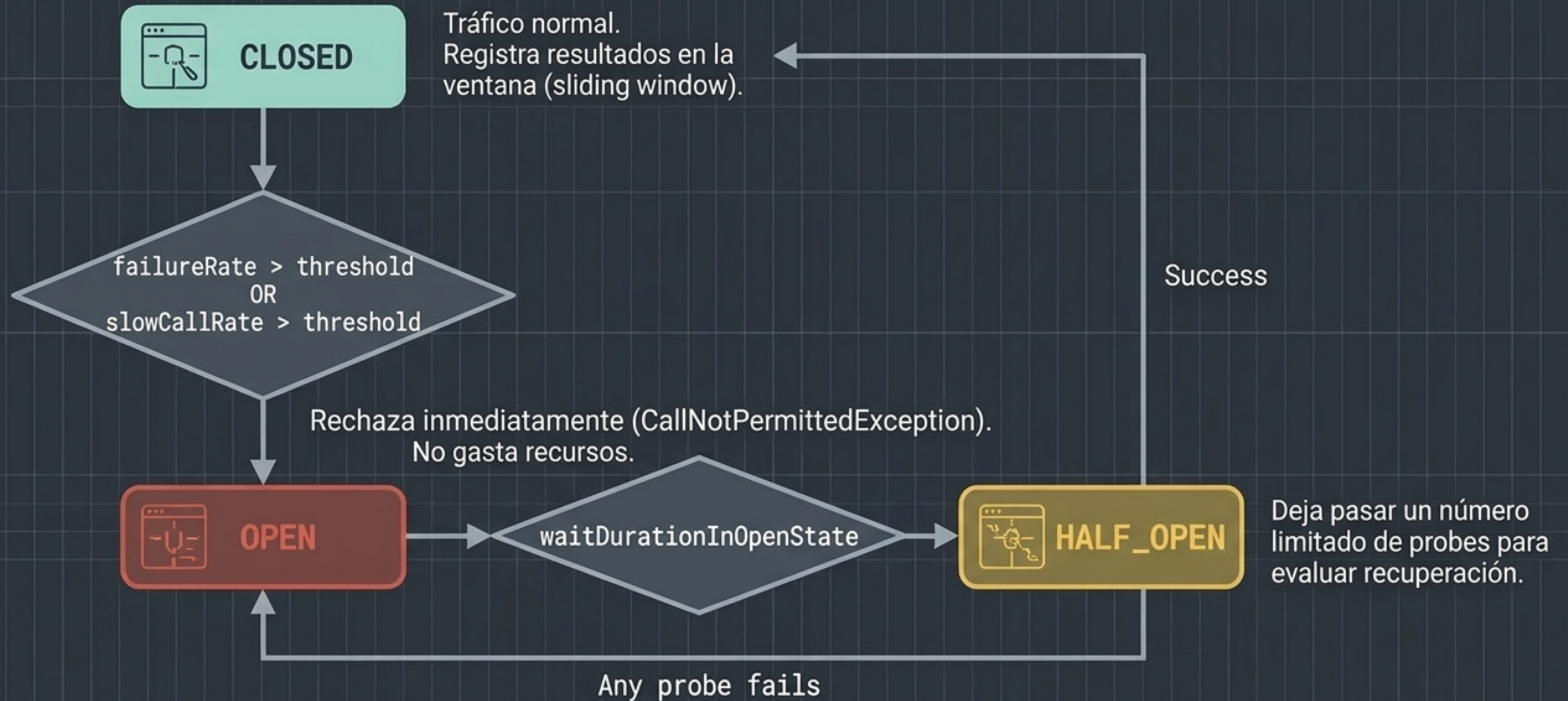
Cada patrón es una respuesta directa a las fallas de la red.

Defensa en profundidad: El orden de los decoradores importa



Este es el orden **efectivo de Resilience4j** para proteger el 95% de los casos de uso.

La anatomía de un Circuit Breaker



Resilience4j añade **DISABLED** (pasa todo) y **FORCED_OPEN** (rechaza todo).

Calibrando los umbrales según la criticidad del servicio

Parámetro	Servicio Crítico	Servicio No Crítico
failureRateThreshold	50%	30%
slowCallRateThreshold	60%	40%
slowCallDurationThreshold	800 ms	500 ms
waitDurationInOpenState	10 s	30 s
permittedNumberOfCallsInHalfOpenState	5	3

Regla de Oro: **waitDurationInOpenState** > tiempo típico de reinicio del downstream.
(De lo contrario, entras en un ciclo infinito OPEN → HALF_OPEN → OPEN).

Implementación I: Spring Boot 4 + Resilience4j

YAML Configuration

```
resilience4j:  
  circuitbreaker:  
    instances:  
      inventoryClient:  
        slidingWindowType: COUNT_BASED  
        slidingWindowSize: 20
```

Java Implementation

```
@TimeLimiter(name = "inventoryClient")  
@CircuitBreaker(name = "inventoryClient",  
    fallbackMethod = "reserveFallback")  
public CompletableFuture<Map> reserve  
    (String sku) {  
    return CompletableFuture.supplyAsync(()  
        -> http.post()...);  
}
```

- resilience4j-spring-boot3 es 100% compatible con Spring Boot 4.
- @TimeLimiter exige tipos asíncronos (CompletableFuture, Mono, Flux).

Implementación II: Quarkus 3 + SmallRye Fault Tolerance

```
@ApplicationScoped
public class PricingClient {
    @Bulkhead(value = 25)
    @Timeout(value = 1500, unit = ChronoUnit.MILLIS)
    @CircuitBreaker(requestVolumeThreshold = 10, failureRatio = 0.5)
    @Retry(maxRetries = 3, delay = 300, jitter = 100)
    @Fallback(fallbackMethod = "fallbackPrice")
    public Price fetch(String sku) { ... }
}
```

Basado en el estándar
MicroProfile FT (portable
a Helidon, Payara).

El nombre del Circuit Breaker
por defecto es
<classname>/<methodName>.

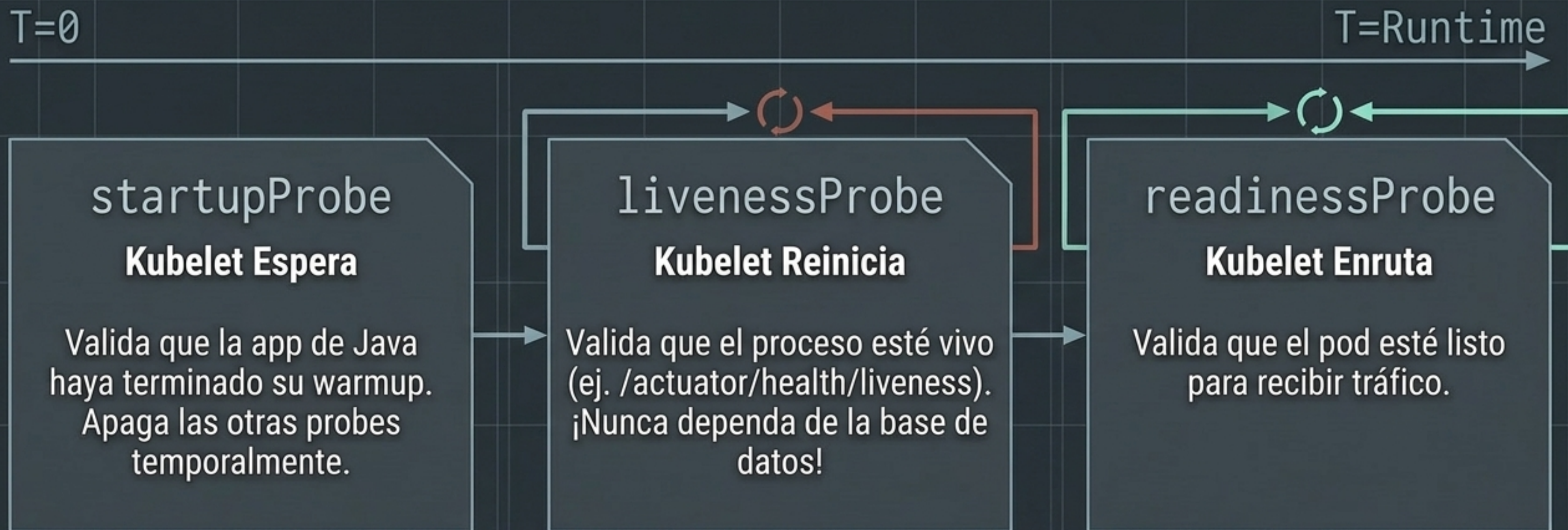
Requiere
quarkus.fault-tolerance.
metrics.enabled=tru
para observabilidad.

Análisis de decisión arquitectónica: Resilience4j vs. SmallRye

Categoría	Spring (Resilience4j)	Quarkus (SmallRye FT)
Especificación	Librería independiente	Estándar MicroProfile
Configuración	Declarativa (<code>application.yml</code>)	Anotaciones en código
Sliding Window	Por tiempo o conteo	Basada en conteo (<code>requestVolumeThreshold</code>)
Slow Calls	Nativo (<code>slowCallRateThreshold</code>)	Requiere extensión (<code>@ApplyGuard</code>)
Métricas	Micrometer nativo	MicroProfile Metrics (Micrometer)

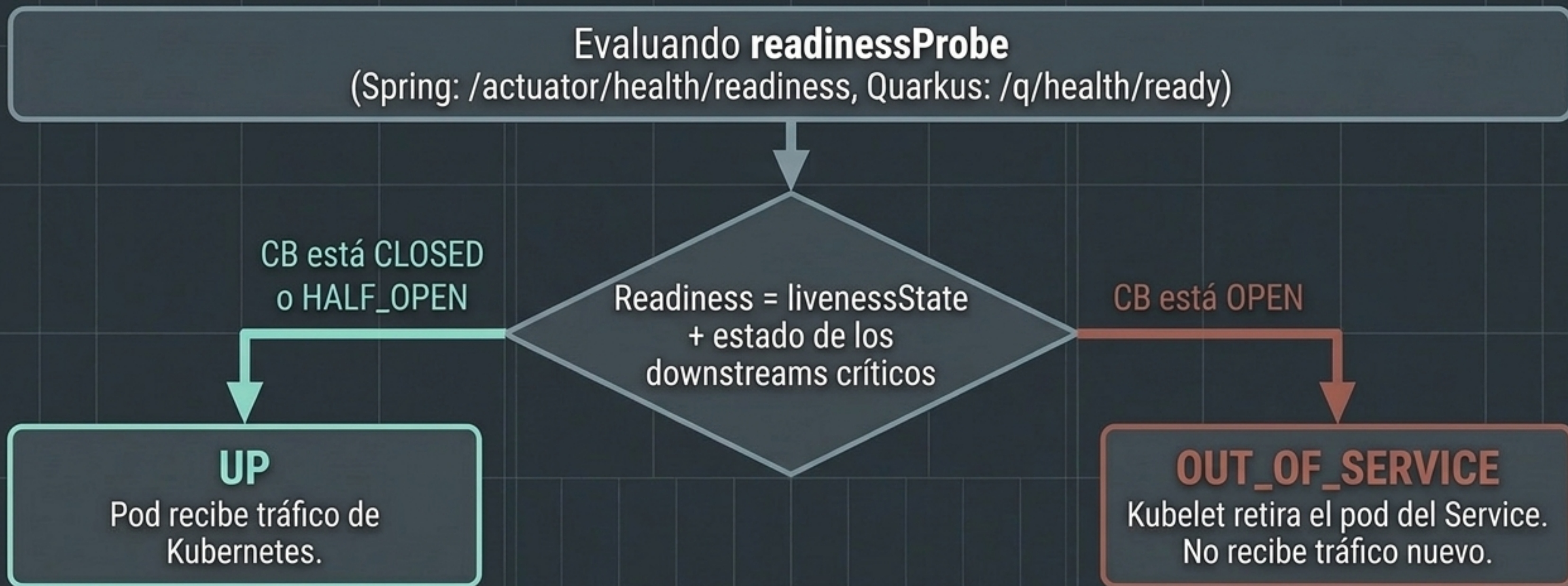
¿Portabilidad de código? → SmallRye.
¿Máxima riqueza de features? → Resilience4j.

La capa de infraestructura: Probes de Kubernetes



Regla de Oro K8s: Liveness reinicia tu pod, Readiness enruta el tráfico. Nunca los mezcles.

Orquestando el caos: Integración de Circuit Breaker y Readiness



Cuidado: Si todos los pods tienen el CB abierto y marcan OUT_OF_SERVICE, K8s devolverá un 503 masivo. Decisión: ¿Prefieres un 503 rápido (fail-fast) o servir respuestas degradadas con fallbacks?

Observabilidad: Las 4 métricas doradas de la resiliencia

CB State

```
resilience4j_circuitbreaker_state{state='open'} == 1
```

1

Alerta inmediata de aislamiento.

Failure Rate

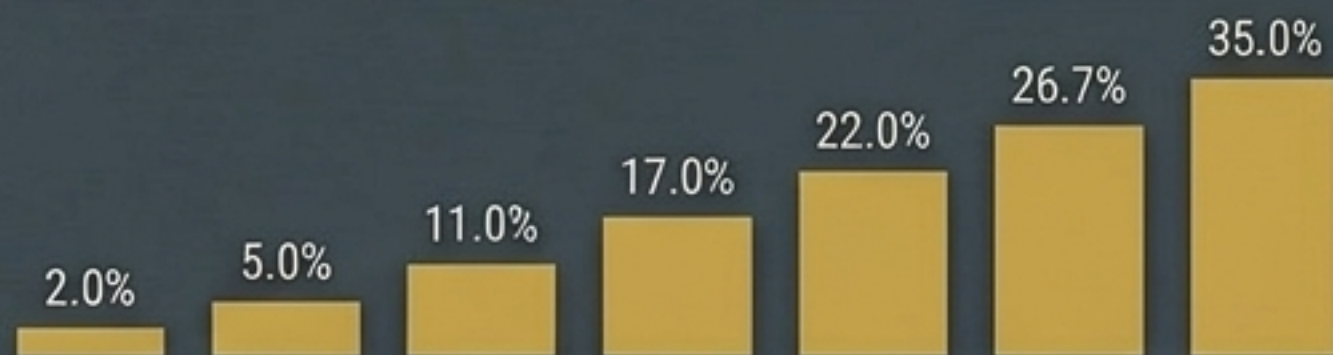
```
rate(resilience4j_circuitbreaker_calls_seconds_count  
{kind='failed'}[5m])
```



Identifica la degradación antes del corte.

Fallback Rate

degraded / total



Mide el % de respuestas con valor reducido.

Bulkhead Saturation

```
resilience4j_bulkhead_available_concurrent_calls == 0
```



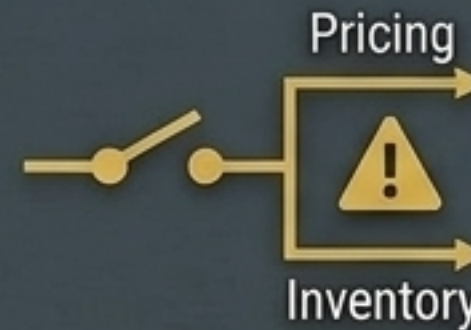
Detecta agotamiento de hilos por latencia remota.

Ilusiones de robustez: Antipatrones comunes



Retry Storm

Reintentos sin backoff. Cuando el downstream sufre, los reintentos simultáneos actúan como un ataque DDoS interno.



El Circuit Breaker Compartido

Un mismo CB para Pricing e Inventory. Si un servicio cae, apagas ambos. Siempre usa un CB por downstream.



Liveness atada a dependencias

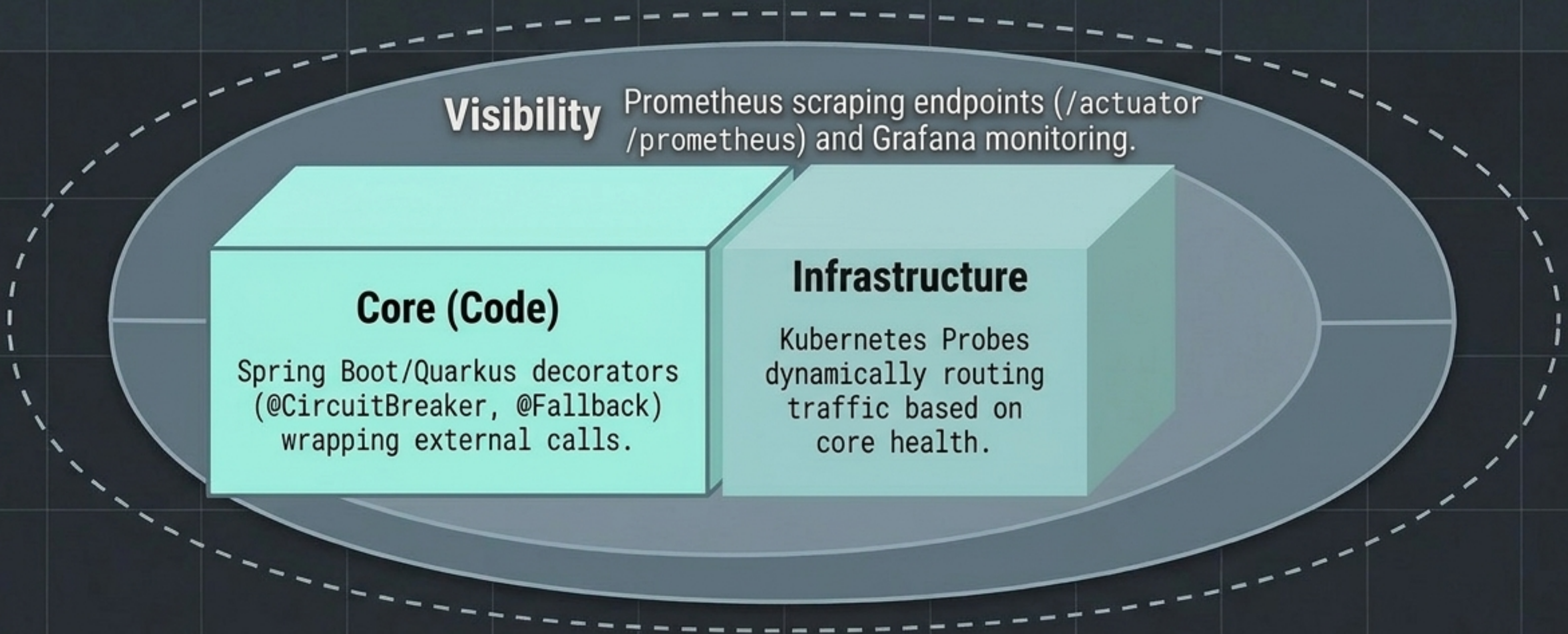
Incluir la conexión de Base de Datos en el livenessProbe. Un blip de red en la DB reinicia toda la flota de pods a la vez -> Outage Total.



Fallback Destructivo

Un fallback que lanza excepciones o llama a otro servicio frágil. El propósito del fallback es devolver algo válido y seguro.

La arquitectura resiliente completa



La resiliencia no es una librería; es una propiedad emergente del sistema completo.

Siguiente frontera: Observabilidad End-to-End con OpenTelemetry (Tracing distribuido de fallos).